Cd TOBB UNIVERSITY OF ECONOMICS AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

BİL 481 – SOFTWARE ENGINEERING PROJECT ASSIGNMENT 2 (PA2 Part 2): DESIGN PATTERNS ADDENDUM

PROJECT NAME: SWAPLY

AUTHORS:
Arda Burak Zararsız
Yusuf Çakır
Yusuf Enes Kütük
Eren Öz
Furkan Yılmaz

DATE: 29.03.2026

Task Matrix

Task	Responsible Member
Problem Identification & Pattern Selection	Eren Öz, Arda Burak Zararsız
Observer Pattern — Design & UML	Yusuf Enes Kütük, Furkan Yılmaz
Repository Pattern — Design & UML	Yusuf Çakır, Eren Öz
Quality Factors & Criteria Analysis	Arda Burak Zararsız, Furkan Yılmaz
Document Formatting & Final Review	Eren Öz, Arda Burak Zararsız

Table of Contents

1. Introduction
2. Design Pattern 1: Observer Pattern
3. Design Pattern 2: Repository Pattern
4. Summary of Design Pattern Decisions

1. Introduction

This document is an addendum to the Swaply Software Design Document (SDD) submitted on 05.03.2026. As the final phase of the design process, two design problems identified within the existing architecture are addressed by applying formal software design patterns. For each pattern, this document explains: (1) the problem it solves, (2) how it is applied in the Swaply architecture, (3) how the UML class diagrams are updated, and (4) the associated design decisions including quality factors and quality criteria.

2. Design Pattern 1: Observer Pattern

2.1 Problem Identified

In the original design, both the Real-Time Chat (FR4) and the Skill-Based Match Notification (FR3) features relied on ad-hoc Firestore onSnapshot calls scattered directly inside screen components (ChatScreen, MapScreen). This created a tight coupling between the UI layer and the data-listening logic. Specifically:

- ChatScreen directly contained the Firestore onSnapshot listener for messages.
- MapScreen independently queried Firestore for nearby matches without any shared notification contract.
- Adding a new screen that also needs to react to match events would require duplicating listener code.
- Unit-testing notification behaviour was impossible without rendering full React Native screen components.

This is a classic 'many screens react to the same data event' problem — precisely the scenario the Observer pattern is designed to solve.

2.2 Pattern Overview

Pattern Name	Observer (Behavioral)
GoF Category	Behavioral
Intent	Define a one-to-many dependency so that when one object changes state, all dependents are notified and updated automatically.
Applied To	FR3 (Match Notifications) and FR4 (Real-Time Chat)

2.3 Application in Swaply Architecture

A new service class, MatchNotificationService, is introduced in the /src/services/ layer. It manages a list of subscribers (observers) and owns the single Firestore onSnapshot listener. UI screen components no longer call Firestore directly; instead they register themselves as observers.

Architecture change summary:

- Before: ChatScreen → Firestore (direct onSnapshot call inside component).
- After: ChatScreen → MatchNotificationService → Firestore (ChatScreen implements IMatchObserver and subscribes to the service).
- The same MatchNotificationService also notifies FCM (push notification dispatch) when a new match document is created, keeping all notification logic centralized.
- MapScreen registers a MapScreenObserver to react to new nearby matches without owning any Firestore listener.

2.4 Updated UML Class Diagram

Class Diagram — Observer Pattern

```
<<interface>> IMatchObserver
+ onMatchFound(matchData): void
+ onMessageReceived(msg): void

MatchNotificationService
- observers: List<IMatchObserver>
- firestoreRef: CollectionRef
+ subscribe(obs): void
+ unsubscribe(obs): void
+ notifyAll(event): void
+ listenToMatches(uid): void
```

Concrete Observers (implement IMatchObserver):

```
ChatScreenObserver
+ onMatchFound(data): void
+ onMessageReceived(msg): void

MapScreenObserver
+ onMatchFound(data): void
+ onMessageReceived(msg): void
```

Relationship: MatchNotificationService uses IMatchObserver (realizes interface). ChatScreenObserver and MapScreenObserver implement IMatchObserver.

2.5 Design Decision

Why Observer was chosen

The Swaply application has multiple screens that must react to the same Firestore events (new messages, new matches). Without a formal pattern, each screen maintains its own listener, leading to resource waste (multiple simultaneous Firestore listeners for the same collection) and code duplication. The Observer pattern provides a single listener that fans out to registered screens, reducing Firestore read costs and isolating the change-detection logic from the UI.

Architectural location

The Subject (MatchNotificationService) lives in /src/services/. The IMatchObserver interface is defined in /src/types/. Concrete observer implementations are co-located with their respective screen files in /src/screens/ to keep screen-specific reaction logic close to the view.

Quality Factor & Criteria

Quality Factor	Metric / Criteria	Target Value
Maintainability	Adding a new screen that reacts to match events requires only implementing IMatchObserver and calling subscribe() — zero changes to existing classes. Cyclomatic complexity of MatchNotificationService target: ≤ 10.	< 10 per function (ESLint)
Performance	Single Firestore onSnapshot listener shared by all observers instead of N separate listeners. Reduces Firestore read operations proportionally to the number of subscribed screens.	≤ 1 active snapshot listener per collection at runtime

Testability	MatchNotificationService can be unit-tested by injecting a mock IMatchObserver and asserting onMatchFound() is called without any React Native rendering.	100% of service methods covered by Jest unit tests

3. Design Pattern 2: Repository Pattern

3.1 Problem Identified

In the original design, Firebase/Firestore API calls (getUserById, findMatches, submitRating, deleteUser) were made directly inside screen components and in the /src/services/firebase.js helper file without a consistent abstraction layer. This caused:

- Screen components (e.g., ProfileScreen, DiscoverScreen) importing Firebase SDK objects directly, creating a dependency on the concrete cloud provider.
- Duplicated Firestore query logic — the same 'find users by skill overlap' query appeared in both the Discover screen and the Map screen.
- No clear contract for what data operations the application requires, making it difficult to swap Firestore for another backend or to mock it during testing.
- FR10 (Account Deletion) required touching multiple files to gather all user-related collections, as there was no single place owning 'all user data operations'.

This violates the Dependency Inversion Principle and makes the data layer difficult to test or change independently of the UI.

3.2 Pattern Overview

Pattern Name	Repository (Architectural / Data Access)
GoF Category	Not a GoF pattern; originates from Domain-Driven Design (DDD)
Intent	Encapsulate all data access logic behind an interface, decoupling domain/business logic from the data source.
Applied To	FR1, FR2, FR7, FR8, FR9, FR10 — all features requiring user data read/write operations.

3.3 Application in Swaply Architecture

An IUserRepository interface is introduced defining all user-related data operations. FirestoreUserRepository implements this interface using the Firebase SDK. Screen components and services now depend only on IUserRepository — never on Firebase directly.

Architecture change summary:

- Before: ProfileScreen → import { db } from '../services/firebase' → db.collection('users').doc(uid).get().
- After: ProfileScreen → IUserRepository.getUserById(uid) — the concrete Firestore implementation is injected at app startup.
- The /src/services/ folder is restructured: firebase.js becomes FirestoreUserRepository.ts implementing IUserRepository defined in /src/types/IUserRepository.ts.
- FR10 deleteUser(uid) now calls a single repository method that atomically deletes the user document, all match documents, and all chat subcollections — previously this was split across three different files.

3.4 UML Class Diagram

Class Diagram — Repository Pattern

```
<<interface>> IUserRepository
+ getUserById(uid): User
+ updateUser(uid, data): void
+ deleteUser(uid): void
+ findMatches(uid): Match[]
+ submitRating(uid, r): void
```

◀—

```
FirestoreUserRepository
- db: FirebaseFirestore
- usersCol: CollectionRef
---------------------------
+ getUserById(uid): User
+ updateUser(uid, data): void
+ deleteUser(uid): void
+ findMatches(uid): Match[]
+ submitRating(uid, r): void
```

Relationship: FirestoreUserRepository implements IUserRepository. Screen services depend only on IUserRepository interface (Dependency Inversion).

3.5 Design Decision

Why Repository was chosen

The Swaply project uses Firebase as its sole backend, but the team acknowledged during design review that Firestore's free Spark Plan has monthly read/write limits. If the project scales or migrates, changing every Firebase call scattered across screens would be prohibitively risky. By placing all data access behind IUserRepository, the migration surface is reduced to a single class: FirestoreUserRepository. The rest of the codebase is completely unaffected.

Additionally, the Repository pattern enables Jest mock implementations of IUserRepository to be injected during unit tests, removing the need to connect to a live Firestore instance for every test — directly supporting the QA Plan's goal of automated unit testing.

Architectural location

IUserRepository interface: /src/types/IUserRepository.ts. FirestoreUserRepository implementation: /src/services/FirestoreUserRepository.ts. Dependency injection entry point: /src/App.tsx (repository instance created once and passed via React Context to all screens).

Quality Factor & Criteria

Quality Factor	Metric / Criteria	Target Value
Maintainability	All Firestore-specific code is isolated to FirestoreUserRepository. Replacing Firestore requires modifying only one file. The interface acts as the stable contract.	0 direct Firebase SDK imports outside /src/services/
Testability	IUserRepository can be mocked in Jest. FR10 (deleteUser) and FR8 (submitRating) can be fully unit-tested without a live Firebase connection.	> 80% code coverage on all repository methods (Jest)
Security (NFR2)	Firebase Security Rules are validated at a single layer. The repository is the only place	0 unauthorized write paths outside FirestoreUserRepository

	allowed to write to Firestore, making security rule auditing straightforward.	

4. Summary of Design Pattern Decisions

Pattern	Problem Solved	Quality Factor	Measurable Criterion
Observer	Tight coupling between UI screens and Firestore listeners; duplicated event handling.	Maintainability, Performance, Testability	≤1 Firestore listener/collection; all service methods Jest-tested.
Repository	Scattered Firebase SDK calls across screens; no abstraction for data access or testing.	Maintainability, Testability, Security	0 direct Firebase imports outside services layer; >80% method coverage.

Both patterns together enforce a clean layered architecture: UI screens depend on abstractions (IMatchObserver, IUserRepository), services own all side effects, and the concrete Firebase implementations are swappable without touching any screen code. This directly satisfies NFR2 (Security), NFR1 (Performance), and the maintainability criteria defined in the QA Plan.